



Introdução a Interfaces



Interfaces em Java

- É uma classe que permite apenas a definição de métodos abstratos (sem implementação) e atributos que serão considerados, implicitamente, como public static final
- A ideia é definir comportamentos que não podem ser implementados de forma genérica, mas você quer forçá-los a existir
- Quando uma classe "herda" de uma interface, diz-se que a implementa e a palavra-chave implements é usada. E nunca diremos que "herda de", mas diremos que "implementa"
- Java permite que uma classe implemente várias interfaces, desde que implemente o código dos métodos definidos em eles (ou a própria classe se torna abstrata)
- Por definição, os métodos de uma interface são públicos, não sendo necessário indicá-lo
- Na prática, funciona como uma classe abstrata onde todos os seus métodos são abstratos

```
public interface Animal {  
    String tipo = "Animal";  
    void comer();  
}
```

- Por sua vez, as interfaces podem herdar de outra interface:

```
public interface Ave extends Animal {  
    void voar();  
}
```



Classes e interfaces

- Se, por exemplo, quiséssemos criar algumas classes que implementassem a interface Animal:

```
public class Gato implements Animal {  
    @Override  
    public void comer() {  
        System.out.println("O gato está comendo");  
    }  
}  
  
public class Cachorro implements Animal {  
    @Override  
    public void comer() {  
        System.out.println("O cachorro está comendo");  
    }  
}
```

E podemos fazer, como fizemos com as classes abstratas:

```
Animal animal = new Gato();  
animal.comer();
```



Herdar e implementar

- Herdar uma interface não nos impede de herdar de uma classe:

```
public class Gato extends Mamifero implements Animal {  
    @Override  
    public void comer() {  
        System.out.println("O gato está comendo");  
    }  
}
```

```
public class Cachorro extends Mamifero implements Animal {  
    @Override  
    public void comer() {  
        System.out.println("o cachorro está comendo");  
    }  
}
```



Implementar várias interfaces

- Como havíamos dito, uma classe pode implementar várias interfaces

```
public class Gato extends Mamifero implements Animal, Mascote {  
    @Override  
    public void comer() {  
        System.out.println("O gato está comendo");  
    }  
    // Métodos da interface Mascote  
}
```



iZoo

- Um zoológico pede que você projete as classes que permitirão definir o modelo de dados para um aplicativo que permite gerenciar todas as informações do Parque Zoológico. É preciso ter em mente que pode haver qualquer tipo de animal, mas todos, afinal, precisam os mesmos cuidados e têm o mesmo comportamento, embora à sua maneira, que pretende se vigiar para garantir o seu bem-estar:
- Quanto aos cuidados tem que ser possível: limpar, alimentar, trocar de quarto e curar alguma doença que eles podem ter
- Quanto ao comportamento, todos eles: vão comer, dormir, se mover, voar (quem puder), botar ovos ou dar para criar uma nova prole de alguma forma. . . .
- Você também deve projetar o modelo de classe pensando que novos animais podem ser incorporados ao Zoológico a qualquer momento. Tente definir este modelo permitindo que seja facilmente adaptável a esta nova situação.
- Definir:
 - **Algumas classes para uma variedade de animais**
 - **Construtores**
 - **Gettes e setters necessários**
 - **Uma classe principal onde testar o funcionamento de todo o modelo projetado**



TokioMarket

- A rede de supermercados TokioMarket contratou você para desenvolver uma aplicação que precisam para gerenciar o pessoal, suas instalações e vendas.
- Deve-se levar em consideração que, em cada loja, existe um gerente, alguns reforçadores e pessoal que trabalha na fila do caixa. Como às vezes há picos de trabalho, mesmo que sejam definidos 3 perfis, todos devem estar preparados para exercer qualquer uma das funções, inclusive o próprio gerente.
- Quanto ao local, existem três tipos: grandes superfícies na periferia da cidade, pequenas instalações no centro e também alguns centros funcionam 24 horas. A diferença entre os três tipos de lojas é o tamanho, a variedade de produtos que oferecem, as horas e o número de pessoas que nelas trabalham.
- Quanto aos clientes, por quererem aumentar a sua lealdade, são sempre questionados se desejam tornar-se parceiros de TokioMarket para dar-lhes um cartão de cliente e assim se qualificar para vários descontos e promoções. Dependendo da despesa mensal que fazem existem 3 perfis: Bronze, Silver e Gold Member. A diferença entre os 3 é que optam por maiores descontos (a percentagem que aplica-se a produtos). Também é verdade que, dependendo do tipo de membro, pode ter descontos em alguns ou outros produtos e Além disso, os Golds são convidados para eventos exclusivos.
- Sobre os produtos, existem várias categorias principais: Casa, Lazer e Trabalho, mas cada produto pode pertencer a vários categorias ao mesmo tempo
- Definir:
 - **Classe para definir todo o problema**
 - **Construtores, Gettes e Setters para todos**
 - **Uma classe principal onde testar o funcionamento de todo o modelo projetado**



Dúvidas?

Tokio.